Ruby Is an Object-Oriented Language

Everything you manipulate in Ruby is an object, and the results of those manipulations are themselves objects.

When you write object-oriented code, you're normally looking to model concepts from the real world. Typically, during this modeling process you discover categories of things that need to be represented. In an online store, the concept of a line item could be such a category. In Ruby, you'd define a *class* to represent each of these categories. You then use this class as a kind of factory that generates *objects*—instances of that class. An object is a combination of state (for example, the quantity and the product ID) and methods that use that state (perhaps a method to calculate the line item's total cost). We'll show how to create classes in *Classes*.

You create objects by calling a *constructor*, a special method associated with a class. The standard constructor is called `new`. Given a class called `LineItem`, you could create line item objects as follows:

```
line_item_one = LineItem.new
line_item_one.quantity = 1
line_item_one.sku      = "AUTO_B_00"
```

You invoke methods by sending a message to an object. The message contains the method's name along with any parameters the method may need. When an object receives a message, it looks into its own class for a corresponding method. Let's look at some method calls:

```
"dave".length
line_item_one.quantity()
cart.add_line_item(next_purchase)
submit_tag "Add to Cart"
```

Parentheses are generally optional in method calls. In Rails applications, you'll find that most method calls involved in larger expressions have